



DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Trabajo práctico

AirMiles

September 13, 2026

Algoritmos y Estructuras de Datos

Grupo5

Integrante	LU	Correo electrónico
Stude, Lautaro	1096/25	studelautaro@gmail.com
Gonçalves, Jazmín	964/24	jazmingoncalves7@gmail.com
Musante, Alejo	1062/25	musantealejo@gmail.com
Marín, Sahara	1340/23	candiotti0317@gmail.com



Facultad de Ciencias Exactas y Naturales

Universidad de Buenos Aires

Ciudad Universitaria - (Pabellón I/Planta Baja)

Intendente Güiraldes 2610 - C1428EGA

Ciudad Autónoma de Buenos Aires - Rep. Argentina

Tel/Fax: (++54 +11) 4576-3300

<http://www.exactas.uba.ar>

enum Categoría ES {Oro, Plata, Bronce, Platino}

Transacción ES struct{tipo: string, millas: $\mathbb{R}$, idOrigen: $\mathbb{Z}$, idDestino: $\mathbb{Z}$,
distancia: $\mathbb{R}$, nombrePromo: string, factorPromo: *factorPromo*}

factorPromo ES $\mathbb{R}$

tope ES $\mathbb{R}$

acumuladasTope ES $\mathbb{R}$

TAD AirMiles {

obs usuarios : conj($\mathbb{Z}$)

obs millasDisponibles : dicc($\mathbb{Z}$, $\mathbb{R}$)

obs millasTotales : dicc($\mathbb{Z}$, $\mathbb{R}$)

obs promocionesConTope : dicc(String, $\langle \text{factorPromo} \times \text{tope} \times \text{acumuladasTope} \rangle$)

obs promocionesSinTope : dicc(String, *factorPromo*)

obs transacciones : dicc($\mathbb{Z}$, seq(*Transacción*))

proc crearSistema() : AirMiles {

requiere { True }

asegura {

$\|res.promocionesSinTope\| = 1 \wedge \text{"NoPromo"} \in res.promocionesSinTope \wedge$

$res.promocionesSinTope[\text{"NoPromo"}] = 1 \wedge$

$\|res.promocionesConTope\| = 0 \wedge$

$res.millasDisponibles = \{\}$ $\wedge$

$res.millasTotales = \{\}$ $\wedge$

$res.usuarios = \langle \rangle \wedge$

$res.transacciones = \{\}$

}

}

proc igualdad(in s1 : AirMiles, in s2 : AirMiles) : bool {

requiere { True }

asegura { $res = true \iff (\forall id : \mathbb{Z}) (id \in s1.usuarios \iff id \in s2.usuarios) \wedge$

$s1.millasDisponibles = s2.millasDisponibles \wedge$

$s1.millasTotales = s2.millasTotales \wedge$

$s1.promocionesConTope = s2.promocionesConTope \wedge$

$s1.promocionesSinTope = s2.promocionesSinTope \wedge$

/* El grupo consideró que dos diccionarios "transacciones" son iguales solo si el orden
de cada transacción (struct) dentro de las secuencias es el mismo */

$s1.transacciones = s2.transacciones \}$

}

proc registrarUsuario(in id : $\mathbb{Z}$, inout sistema : AirMiles) {

requiere { $sistema = sistema_0 \wedge \neg esUsuario(id, sistema) \wedge id > 0 \}$

asegura {

$sistema.usuarios = sistema_0.usuarios \cup \{id\} \wedge$

$sistema.millasDisponibles = setKey(sistema_0.millasDisponibles, id, 0) \wedge$

$sistema.millasTotales = setKey(sistema_0.millasTotales, id, 0) \wedge$

$sistema.transacciones = setKey(sistema_0.transacciones, id, \langle \rangle) \wedge$

$sistema.promocionesSinTope = sistema_0.promocionesSinTope \wedge$

$sistema.promocionesConTope = sistema_0.promocionesConTope$

}

}

```

proc registrarUsuarioConReferidor(in id :  $\mathbb{Z}$ , in referidor :  $\mathbb{Z}$ , inout sistema : AirMiles)
{
  requiere { sistema = sistema0  $\wedge$   $\neg$ esUsuario(id, sistema)  $\wedge$ 
    esUsuario(referidor, sistema)  $\wedge$  id > 0 }
  asegura {
    sistema.usuarios = sistema0.usuarios  $\cup$  {id}  $\wedge$ 
    sistema.millasTotales[id] = 500  $\wedge$ 
    sistema.millasDisponibles[id] = 500  $\wedge$ 
    sistema.millasTotales[referidor] = sistema0.millasTotales[referidor] + 500  $\wedge$ 
    sistema.millasDisponibles[referidor] = sistema0.millasDisponibles[referidor] +
    500  $\wedge$ 
    sistema.promocionesSinTope = sistema0.promocionesSinTope  $\wedge$ 
    sistema.promocionesConTope = sistema0.promocionesConTope  $\wedge$ 
    /* El grupo tom3 la decisi3n de registrar en transacciones el idOrigen y el
    idDestino. En casos como en acumularMillas, donde sucede que una o ambas
    categorías no tienen representaci3n, optamos por darle valor -1, ya que no son
    usuarios por requisito de registrarUsuario y tienen un tipo que es distinto
    de Transferencia. */
    sistema.transacciones[id] =  $\langle\langle$ tipo: "bono", millas: 500, idOrigen: -1,
    idDestino: -1, distancia: 0, nombrePromo: "NoPromo", factorPromo: 1 $\rangle\rangle$   $\wedge$ 
    sistema.transacciones[referidor] = sistema0.transacciones[referidor] + +
     $\langle\langle$ tipo: "bono", millas: 500, idOrigen: -1, idDestino: -1, distancia: 0,
    nombrePromo: "NoPromo", factorPromo: 1 $\rangle\rangle$   $\wedge$ 
    diccionariosIgualesIgnorandoIDS({id, referidor}, sistema0, sistema)
  }
}

```

```

proc acumularMillas(in id :  $\mathbb{Z}$ , in distancia :  $\mathbb{R}$ , in promo : String, inout sistema :
AirMiles) {
  requiere { sistema = sistema0  $\wedge$  esUsuario(id, sistema)  $\wedge$  distancia > 0  $\wedge$ 
    (promo  $\in$  sistema.promocionesSinTope  $\vee$  promo  $\in$  sistema.promocionesConTope) }
  asegura {
    sistema.usuarios = sistema0.usuarios  $\wedge$ 
    diccionariosIgualesIgnorandoIDS({id}, sistema0, sistema)
  }
  asegura {
    promo  $\in$  sistema0.promocionesSinTope  $\rightarrow_L$ 
    (sistema.millasTotales[id] = sistema0.millasTotales[id] +
    (factorCategoria(id, sistema0) * distancia * sistema0.promocionesSinTope[promo])  $\wedge$ 
    sistema.millasDisponibles[id] = sistema0.millasDisponibles[id] +
    (factorCategoria(id, sistema0) * distancia * sistema0.promocionesSinTope[promo])  $\wedge$ 
    sistema.promocionesConTope = sistema0.promocionesConTope  $\wedge$ 
    sistema.promocionesSinTope = sistema0.promocionesSinTope  $\wedge$ 
    sistema.transacciones[id] = sistema0.transacciones[id] + +
     $\langle\langle$ tipo: "vuelo", millas: factorCategoria(id, sistema0) * distancia *
    sistema0.promocionesSinTope[promo], idOrigen: id, idDestino: -1, distancia: distancia,
    nombrePromo: promo, factorPromo: sistema0.promocionesSinTope[promo] $\rangle\rangle$ )
  }
  asegura {

```

```

    promo ∈ sistema0.promocionesConTope →L
    (sistema.millasTotales[id] = sistema0.millasTotales[id] +
    cuentaParaActualizarMillas(id, distancia, promo, sistema0) ∧
    sistema.millasDisponibles[id] = sistema0.millasDisponibles[id] +
    cuentaParaActualizarMillas(id, distancia, promo, sistema0) ∧ sistema.transacciones[id] =
    sistema0.transacciones[id] + +
    ⟨⟨tipo: “vuelo”, millas: cuentaParaActualizarMillas(id, distancia, promo, sistema0),
    idOrigen: id, idDestino: -1, distancia: distancia, nombrePromo: promo,
    factorPromo: sistema0.promocionesConTope[promo]0⟩⟩ ∧ sistema.promocionesSinTope =
    sistema0.promocionesSinTope ∧ (millasGanadasPromo(id, distancia, promo, sistema0) ≥
    sistema0.promocionesConTope[promo]1 - sistema0.promocionesConTope[promo]2 →L
    sistema.promocionesConTope = delKey(sistema0.promocionesConTope, promo)) ∧
    (millasGanadasPromo(id, distancia, promo, sistema0) <
    sistema0.promocionesConTope[promo]1 - sistema0.promocionesConTope[promo]2 →L
    sistema.promocionesConTope =
    setKey(sistema0.promocionesConTope, promo, ⟨sistema0.promocionesConTope[promo]0,
    sistema0.promocionesConTope[promo]1, sistema0.promocionesConTope[promo]2 +
    millasGanadasPromo(id, distancia, promo, sistema0)⟩⟩))
  }
}
proc canjearMillas(in id : ℤ, in millasACanjear : ℝ, inout sistema : AirMiles) {
  requiere { sistema = sistema0 ∧ esUsuario(id, sistema) ∧
  millasACanjear > 0 ∧ sistema.millasDisponibles[id] ≥ millasACanjear }
  asegura {
    sistema.usuarios = sistema0.usuarios ∧
    sistema.promocionesSinTope = sistema0.promocionesSinTope ∧
    sistema.promocionesConTope = sistema0.promocionesConTope ∧
    sistema.millasTotales = sistema0.millasTotales ∧
    sistema.millasDisponibles[id] = sistema0.millasDisponibles[id] - millasACanjear ∧
    sistema.transacciones[id] = sistema0.transacciones[id]
    ⟨⟨tipo: “canje”, millas: millasACanjear, idOrigen: id,
    idDestino: -1, distancia: 0, nombrePromo: ””, factorPromo: 0⟩⟩ ∧
    diccionariosIgualesIgnorandoIDS({id}, sistema0, sistema)
  }
}
proc transferirMillas(in idOrigen : ℤ, in idDestino : ℤ, in millasATransferir : ℝ, inout sistema :
AirMiles) {
  requiere { sistema = sistema0 ∧ esUsuario(idOrigen, sistema) ∧
  esUsuario(idDestino, sistema) ∧ idOrigen ≠ idDestino ∧
  millasATransferir > 0 ∧ sistema.millasDisponibles[idOrigen] ≥ millasATransferir
  }
  asegura {
    sistema.usuarios = sistema0.usuarios ∧
    sistema.promocionesSinTope = sistema0.promocionesSinTope ∧
    sistema.promocionesConTope = sistema0.promocionesConTope ∧
    sistema.millasTotales[idOrigen] =
    sistema0.millasTotales[idOrigen] - millasATransferir ∧
    sistema.millasTotales[idDestino] =
    sistema0.millasTotales[idDestino] + millasATransferir ∧

```

```

    sistema.millasDisponibles[idOrigen] = sistema0.millasDisponibles[idOrigen] -
    millasATransferir ∧
    sistema.millasDisponibles[idDestino] = sistema0.millasDisponibles[idDestino] +
    millasATransferir ∧
    sistema.transacciones[idOrigen] = sistema0.transacciones[idOrigen]
    ⟨⟨tipo: "transferencia", millas: millasATransferir,
    idOrigen: idOrigen, idDestino: idDestino, distancia: 0,
    nombrePromo: "", factorPromo: 0⟩⟩ ∧
    sistema.transacciones[idDestino] = sistema0.transacciones[idDestino]
    ⟨⟨tipo: "transferencia", millas: millasATransferir,
    idOrigen: idOrigen, idDestino: idDestino, distancia: 0,
    nombrePromo: "", factorPromo: 0⟩⟩ ∧
    diccionariosIgualesIgnorandoIDS({idOrigen, idDestino}, sistema0, sistema)
  }
}
}
proc consultarSaldo(in id : ℤ, in sistema : AirMiles) : ℝ {
  requiere { esUsuario(id, sistema) }
  asegura { res = sistema.millasDisponibles[id] }
}
proc consultarCategoría(in id : ℤ, in sistema : AirMiles) : Categoria {
  requiere { esUsuario(id, sistema) }
  asegura {
    (res = Bronce → esBronce(id, sistema)) ∧
    (res = Plata → esPlata(id, sistema)) ∧
    (res = Oro → esOro(id, sistema)) ∧
    (res = Platino → esPlatino(id, sistema))
  }
}
}
proc rankingMillas(in N : ℤ, in sistema : AirMiles) : seq⟨ℤ⟩ {
  requiere { N > 0 }
  asegura {
    (N ≤ ||sistema.usuarios|| → ||res|| = N) ∧
    (N > ||sistema.usuarios|| → ||res|| = ||sistema.usuarios||) ∧
    (∀i : ℤ) (0 ≤ i < ||res|| →L res[i] ∈ sistema.usuarios) ∧
    (∀i, j : ℤ) (0 ≤ i < ||res|| ∧ 0 ≤ j < ||res|| ∧ i ≠ j →L
    res[i] ≠ res[j]) ∧
    estaOrdenadoPorMillas(res, sistema) ∧
    (∀i : ℤ) (0 ≤ i < ||res|| →L
    (∀u : ℤ) (u ∈ sistema.usuarios ∧ u ∉ res →L
    sistema.millasDisponibles[res[i]] ≥ sistema.millasDisponibles[u]))
  }
}
}
proc resumenTransacciones(in id : ℤ, in sistema : AirMiles) : ⟨ℝ, ℝ, ℝ⟩ {
  requiere { esUsuario(id, sistema) }
  asegura {
    res0 = sumarMillasPorTipo(sistema.transacciones[id], "vuelo") ∧
    res1 = sumarMillasPorTipo(sistema.transacciones[id], "canje") ∧
    res2 = sumarMillasPorTipo(sistema.transacciones[id], "transferencia")
  }
}

```

```

}
proc iniciarPromocion(in nombre : String, in factorPromo :  $\mathbb{R}$ , inout sistema : AirMiles)
{
  requiere { sistema = sistema0  $\wedge$  factorPromo  $\geq 1 \wedge$ 
  nombre  $\notin$  sistema.promocionesSinTope }
  asegura {
    sistema.promocionesSinTope =
    setKey(sistema0.promocionesSinTope, nombre, factorPromo)  $\wedge$ 
    sistema.usuarios = sistema0.usuarios  $\wedge$ 
    sistema.millasTotales = sistema0.millasTotales  $\wedge$ 
    sistema.millasDisponibles = sistema0.millasDisponibles  $\wedge$ 
    sistema.transacciones = sistema0.transacciones  $\wedge$ 
    sistema.promocionesConTope = sistema0.promocionesConTope
  }
}
proc iniciarPromocionConTope(in nombreDePromocion : String, in factorPromo :  $\mathbb{R}$ , in tope :
 $\mathbb{R}$ , inout sistema : AirMiles) {
  requiere { sistema = sistema0  $\wedge$  nombreDePromocion  $\notin$  sistema.promocionesConTope }
  asegura {
    sistema.promocionesConTope = setKey(sistema0.promocionesConTope,
    nombreDePromocion,  $\langle$ factorPromo, tope, 0 $\rangle$ )  $\wedge$ 
    sistema.usuarios = sistema0.usuarios  $\wedge$ 
    sistema.millasTotales = sistema0.millasTotales  $\wedge$ 
    sistema.millasDisponibles = sistema0.millasDisponibles  $\wedge$ 
    sistema.transacciones = sistema0.transacciones  $\wedge$ 
    sistema.promocionesSinTope = sistema0.promocionesSinTope
  }
}
proc finalizarPromocion(in promo : String, inout sistema : AirMiles) {
  requiere { sistema = sistema0  $\wedge$ 
  (promo  $\in$  sistema.promocionesSinTope  $\vee$  promo  $\in$  sistema.promocionesConTope)
 $\wedge$  promo  $\neq$  "noPromo" }
  asegura {
    (promo  $\in$  sistema.promocionesSinTope  $\rightarrow_L$ 
    sistema.promocionesSinTope = delKey(sistema0.promocionesSinTope, promo)  $\wedge$ 
    sistema.promocionesConTope = sistema0.promocionesConTope)
 $\wedge$ 
    (promo  $\in$  sistema.promocionesConTope  $\rightarrow_L$ 
    sistema.promocionesConTope = delKey(sistema0.promocionesConTope, promo)  $\wedge$ 
    sistema.promocionesSinTope = sistema0.promocionesSinTope)  $\wedge$ 
    sistema.usuarios = sistema0.usuarios  $\wedge$ 
    sistema.millasTotales = sistema0.millasTotales  $\wedge$ 
    sistema.millasDisponibles = sistema0.millasDisponibles  $\wedge$ 
    sistema.transacciones = sistema0.transacciones
  }
}
proc elQueTieneMasMillas(in sistema : AirMiles) :  $\mathbb{Z}$  {
  requiere { sistema.usuarios  $\neq \langle \rangle$  }
}

```

```

    asegura {
      res ∈ sistema.usuarios ∧
      (∀id : ℤ) (id ∈ sistema.usuarios →L
        sistema.millasTotales[res] ≥ sistema.millasTotales[id])
    }
  }
}
proc elMasCanjeador(in sistema : AirMiles) : ℤ {
  requiere { sistema.usuarios ≠ ⟨⟩ }
  asegura {
    res ∈ sistema.usuarios ∧
    (∀id : ℤ) (id ∈ sistema.usuarios →L
      millasCanjeadas(res, sistema) ≥ millasCanjeadas(id, sistema))
  }
}
proc ParesTransferidoresExclusivos() : seq⟨ℤ × ℤ⟩ {
  requiere { True }
  asegura {
    (∀id, id' : ℤ) ((id, id') ∈ res ⇔ id ∈ sistema.usuarios ∧
      id' ∈ sistema.usuarios ∧ soloLeTransfirio(id, id') ∧ soloLeTransfirio(id', id)) ∧
    noHayRepetidos(res)
  }
}

pred esUsuario(id : ℤ, sistema : AirMiles) { id ∈ sistema.usuarios }

pred esBronce(id : ℤ, sistema : AirMiles) { sistema.millasTotales[id] < 10000 }

pred esPlata(id : ℤ, sistema : AirMiles) {
  sistema.millasTotales[id] ≥ 10000 ∧ sistema.millasTotales[id] < 50000 }

pred esOro(id : ℤ, sistema : AirMiles) {
  sistema.millasTotales[id] ≥ 50000 ∧ sistema.millasTotales[id] < 1000000 }

pred esPlatino(id : ℤ, sistema : AirMiles) { sistema.millasTotales[id] ≥ 1000000 }

aux factorCategoria(id : ℤ, sistema : AirMiles) : ℝ = ifThenElse(esBronce(id, sistema), 1,
  ifThenElse(esPlata(id, sistema), 1.5,
  ifThenElse(esOro(id, sistema), 2, 0)))

pred millasTotalesOtrosIguales(idsIgnorados : conj⟨ℤ⟩, sistema : AirMiles, sistema' :
  AirMiles) {
  (∀id' : ℤ) (id' ∈ sistema.usuarios ∧ id' ∉ idsIgnorados →
    sistema.millasTotales[id'] = sistema'.millasTotales[id'])
}

pred millasDisponiblesOtrosIguales(idsIgnorados : conj⟨ℤ⟩, sistema : AirMiles, sistema' :
  AirMiles) {
  (∀id' : ℤ) (id' ∈ sistema.usuarios ∧ id' ∉ idsIgnorados →
    sistema.millasDisponibles[id'] = sistema'.millasDisponibles[id'])
}

```

```

pred transaccionesOtrosIguales(idsIgnorados : conj⟨ℤ⟩, sistema : AirMiles, sistema' :
AirMiles) {
  (∀id' : ℤ) (id' ∈ sistema.usuarios ∧ id' ∉ idsIgnorados →
    sistema.transacciones[id'] = sistema'.transacciones[id'])
}

pred diccionariosIgualesIgnorandoIDS(idsIgnorados : conj⟨ℤ⟩, sistema : AirMiles, sistema' :
AirMiles) {
  /* Chequea que los diccionarios se mantengan iguales en todas las posiciones
  excepto en las indicadas por los parámetros, se decidió reemplazar setKey por claridad. */
  millasTotalesOtrosIguales(idsIgnorados, sistema, sistema') ∧
  millasDisponiblesOtrosIguales(idsIgnorados, sistema, sistema') ∧
  transaccionesOtrosIguales(idsIgnorados, sistema, sistema')
}

pred estaOrdenadoPorMillas(s : seq⟨ℤ⟩, sistema : AirMiles) {
  (∀i : ℤ) (0 ≤ i < ||s|| - 1 →L
    sistema.millasDisponibles[s[i]] ≥ sistema.millasDisponibles[s[i + 1]])
}

aux sumarMillasPorTipo(transaccionesUsuario : seq⟨Transacción⟩, t : String) : ℝ =
  ∑i=0||transaccionesUsuario||-1 ifThenElse(transaccionesUsuario[i].tipo = t,
    transaccionesUsuario[i].millas, 0)

pred millasTotalesMayoresADisponibles(sistema : AirMiles) {
  (∀id : ℤ) (id ∈ sistema.usuarios →
    sistema.millasTotales[id] ≥ sistema.millasDisponibles[id])
}

pred soloLeTransfirió(id : ℤ, id' : ℤ, sistema : AirMiles) {
  /* Dados dos usuarios, verifica que el primero solo haya recibido o enviado
  una transferencia al segundo. */
  (∀i : ℤ) (0 ≤ i ≤ ||sistema.transacciones[id]|| - 1 ∧L
    sistema.transacciones[id][i].tipo = "transferencia"
    (sistema.transacciones[id][i].idOrigen = id' ∨
      sistema.transacciones[id][i].idDestino = id')) ∧
  (∃j : ℤ) (0 ≤ j ≤ ||sistema.transacciones[id']|| - 1 ∧L
    sistema.transacciones[id'][j].tipo =
      "transferencia")
}

aux devuelveMinimo(a, b : ℤ) : ℤ = IfThenElse(a ≥ b, b, a)

pred noHayRepetidos(lista : seq⟨⟨ℤ × ℤ⟩⟩) {
  (∀i, j : ℤ) (0 ≤ i < j < ||lista|| →L
    (lista[i]0 ≠ lista[j]0 ∨ lista[i]1 ≠ lista[j]1) ∧
    (lista[i]0 ≠ lista[j]1 ∨ lista[i]1 ≠ lista[j]0))
}

aux MillasCanjeadas(id : ℤ, sistema : AirMiles) : ℝ = millasUtilizadas(id, sistema) +
  millasTransferidas(id, sistema)

```

aux millasUtilizadas(*id* : $\mathbb{Z}$, *sistema* : AirMiles) : $\mathbb{R}$ = *sistema.millasTotales*[*id*] – *sistema.millasDisponibles*[*id*]

aux millasTransferidas(*id* : $\mathbb{Z}$, *sistema* : AirMiles) : $\mathbb{R}$ =
 $\sum_{i=0}^{\|sistema.transacciones[id]\| - 1} \text{ifThenElse}(id = sistema.transacciones[id][i].idOrigen \wedge$
sistema.transacciones[*id*][*i*].*idDestino* > 0, *sistema.transacciones*[*id*][*i*].*millas*, 0)

aux millasGanadasPromo(*id* : $\mathbb{Z}$, *distancia* : $\mathbb{R}$, *promo* : String, *sistema* : AirMiles) : $\mathbb{R}$ =
factorCategoria(*id*, *sistema*) * *distancia* * (*sistema.promocionesConTope*[*promo*]₀ – 1)

aux cuentaParaActualizarMillas(*id* : $\mathbb{Z}$, *distancia* : $\mathbb{R}$, *promo* : String, *sistema* : AirMiles) :
 $\mathbb{R}$ = *ifThenElse*(*millasGanadasPromo*(*id*, *distancia*, *promo*, *sistema*) $\geq$
sistema.promocionesConTope[*promo*]₁ – *sistema.promocionesConTope*[*promo*]₂,
devuelveMinimo(*sistema.promocionesConTope*[*promo*]₁ –
sistema.promocionesConTope[*promo*]₂, *millasGanadasPromo*(*id*, *distancia*, *promo*, *sistema*)) +
factorCategoria(*id*, *sistema*) * *distancia*, *millasGanadasPromo*(*id*, *distancia*, *promo*, *sistema*) +
factorCategoria(*id*, *sistema*) * *distancia*)
}